



```
In [ ]: PAWAR SAKSHI MAHENDRA  
        ROLL NO - 49  
        PRACTICAL NO - 10
```

```
In [1]: import numpy as np  
import tensorflow as tf  
from tensorflow.keras.datasets import cifar10  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense, Dropout, Flatten, Conv2D, MaxPooling2D  
from tensorflow.keras.optimizers import SGD  
from tensorflow.keras.preprocessing.image import ImageDataGenerator  
from tensorflow.keras.utils import to_categorical
```

```
In [2]: (X_train, y_train), (X_test, y_test) = cifar10.load_data()  
  
y_train = to_categorical(y_train, 10)  
y_test = to_categorical(y_test, 10)
```

```
In [3]: model = Sequential()  
  
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))  
model.add(Conv2D(32, (3, 3), activation='relu'))  
model.add(MaxPooling2D(pool_size=(2, 2)))  
model.add(Dropout(0.25))  
  
model.add(Conv2D(64, (3, 3), activation='relu'))  
model.add(Conv2D(64, (3, 3), activation='relu'))  
model.add(MaxPooling2D(pool_size=(2, 2)))  
model.add(Dropout(0.25))  
  
model.add(Flatten())  
model.add(Dense(512, activation='relu'))  
model.add(Dropout(0.5))  
model.add(Dense(10, activation='softmax'))
```

C:\Users\saksh\anaconda3\Lib\site-packages\keras\src\layers\convolutional\basic_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
In [4]: sgd = SGD(learning_rate=0.01, momentum=0.9, nesterov=True)  
  
model.compile(loss='categorical_crossentropy',  
              optimizer=sgd,  
              metrics=['accuracy'])
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

```
In [5]: train_datagen = ImageDataGenerator(  
        rescale=1./255,
```

```

        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True
    )

    test_datagen = ImageDataGenerator(rescale=1./255)

    train_set = train_datagen.flow(X_train, y_train, batch_size=32)
    test_set = test_datagen.flow(X_test, y_test, batch_size=32)

```

```

In [6]: model.fit(
        train_set,
        steps_per_epoch=len(X_train)//32,
        epochs=10,    # keep small first
        validation_data=test_set,
        validation_steps=len(X_test)//32
    )

```

Epoch 1/10

1562/1562 ————— **139s** 86ms/step - accuracy: 0.3135 - loss: 1.8596
 - val_accuracy: 0.4725 - val_loss: 1.4368

Epoch 2/10

1/1562 ————— **1:11** 46ms/step - accuracy: 0.3750 - loss: 1.9583

C:\Users\saksh\anaconda3\Lib\site-packages\keras\src\trainers\epoch_iterator.py:116: UserWarning: Your input ran out of data; interrupting training. Make sure that your dataset or generator can generate at least `steps\_per\_epoch \* epochs` batches. You may need to use the `.repeat()` function when building your dataset.

self._interrupted_warning()

1562/1562  **6s** 4ms/step - accuracy: 0.3750 - loss: 1.9583 - val_accuracy: 0.4741 - val_loss: 1.4419
Epoch 3/10
1562/1562  **129s** 82ms/step - accuracy: 0.4612 - loss: 1.4791 - val_accuracy: 0.5288 - val_loss: 1.2999
Epoch 4/10
1562/1562  **6s** 4ms/step - accuracy: 0.6250 - loss: 0.9908 - val_accuracy: 0.5233 - val_loss: 1.3119
Epoch 5/10
1562/1562  **130s** 83ms/step - accuracy: 0.5210 - loss: 1.3361 - val_accuracy: 0.5775 - val_loss: 1.1755
Epoch 6/10
1562/1562  **6s** 4ms/step - accuracy: 0.5312 - loss: 1.3204 - val_accuracy: 0.5716 - val_loss: 1.1829
Epoch 7/10
1562/1562  **172s** 110ms/step - accuracy: 0.5587 - loss: 1.2384 - val_accuracy: 0.6087 - val_loss: 1.0962
Epoch 8/10
1562/1562  **10s** 6ms/step - accuracy: 0.6250 - loss: 1.2231 - val_accuracy: 0.6262 - val_loss: 1.0562
Epoch 9/10
1562/1562  **241s** 154ms/step - accuracy: 0.5869 - loss: 1.1624 - val_accuracy: 0.6573 - val_loss: 0.9742
Epoch 10/10
1562/1562  **9s** 6ms/step - accuracy: 0.6562 - loss: 1.0547 - val_accuracy: 0.6595 - val_loss: 0.9713

Out[6]: <keras.src.callbacks.history.History at 0x1cf96d0acf0>

```
In [7]: score = model.evaluate(test_set, verbose=0)

        print("Test loss:", score[0])
        print("Test accuracy:", score[1])
```

Test loss: 0.9712466597557068
Test accuracy: 0.659500002861023

In []: